

EvoODE — Projektjournal

Stand: 2026-07-23 · Zeitraum: 2026-04-20 bis 2026-07-23

0. Wie dieses Dokument zu lesen ist

Dieses Journal ist der **rote Faden**: Es erzählt, was gebaut wurde, was verworfen wurde, warum, und was die Zahlen dazu sagen. Es ersetzt keines der bestehenden Dokumente, sondern verbindet sie:

Dokument	Rolle
CLAUDE.md	Zustandsbeschreibung: Architektur, Konventionen, aktuelle Prioritäten
DIARY.md	Chronologisches Protokoll, neueste Einträge zuerst, mit Commit-Hashes
PAPER_1.md	Ausführungsplan für Paper 1, maßgeblich bei Widersprüchen
dieses Journal	Narrativ und Begründungen: der Zusammenhang zwischen den Einträgen

Alle Zahlen stammen aus protokollierten Messungen. Wo eine Aussage eine Vermutung ist, steht das ausdrücklich dabei.

1. Das Projekt in einem Absatz

EvoODE ist ein Julia-Forschungsframework zur datengetriebenen Entdeckung interpretierbarer ODE-Systeme. Der Ansatz unterscheidet sich von SINDy (feste Bibliothek, direkte Regression) und von klassischer genetischer Programmierung (globale Suche aus großen Zufallsstrukturen): EvoODE **beginnt minimal und wächst inkrementell**, und erhöht Komplexität nur, wenn einfachere Strukturen nicht ausreichen. Der wissenschaftliche Kern ist nicht das Fitten von ODEs, sondern die Frage, wie Modellstrukturen aufgebaut, erweitert, validiert und kontrolliert werden sollten.

Forschungsfokus der Dissertation: **effiziente und robuste Suchstrategien für die interpretierbare Entdeckung gekoppelter ODE-Systeme.**

2. Der Zeitstrahl in Phasen

Zeitraum	Phase	Ergebnis
2026-04-20 – 04-22	Fundament	Kernpipeline, EvoGrow v1/v2.1/v2.2, GP-Baseline, Experimentinfrastruktur
2026-04-22 – 05-08	Phase-A-Experiment	300 Läufe, 4,5 Tage Rechenzeit, 300/300 erfolgreich
2026-05-08 – 05-11	Auswertung & Freeze	H1 partial, H2 supported, H3 partial, H4 vakuös
2026-05-17	Strategischer Pivot	Paper 1 wird kein Pretuning-Paper; Gate-Struktur eingeführt
2026-05-30	Gate 1	v2.2 nicht paper-ready → EvoGrow v3 ausgelöst
2026-07-20	v3-Entwurf + Regressionshistorie	Designnotiz, Lockstep-Brücke, append-only Historie
2026-07-22	Kostenzusammenbruch	Volllauf abgebrochen; 62 % der Rechenzeit oberhalb der nötigen Stage
2026-07-22 – 07-23	Screening & Numerik	Screening verworfen, wieder aufgenommen; Numerikbefund; Faktor 6,2

3. Die Chronik

3.1 Fundament (2026-04-20 bis 2026-04-22)

Die Kernpipeline `discover()` steht: Daten → Struktur → Parameter → Simulation → Bewertung → Iteration. Implementiert wurden EvoGrow (v1 flach, v2.1 gestuft, v2.2 stage-lokal), eine GP-Baseline, `StagedPolynomialBasis` mit fünf Komplexitätsstufen (linear, selbst-quadratisch, paarweise Kreuzterme, selbst-kubisch, trigonometrisch), `MSELoss` und `BFGSOptimizer`.

Zwei Designachsen wurden bewusst getrennt gehalten und dürfen nicht wieder zusammengelegt werden:

- **Stage Progression Policy** — wann wird eine Stage gehalten, promoviert oder beendet
- **Stage Usage Policy** — wie stark werden neu freigeschaltete Terme danach bevorzugt (`:hard` , `:soft` , `:passive`)

Ebenfalls hier entstanden: `pretune.jl` (Least-Squares-Warmstart über finite Differenzen) und die Experimentinfrastruktur mit Manifest-Generator, sequentiellm Runner und Aggregator, inklusive Per-Run-Dateiprotokoll mit atomaren Writes.

3.2 Das Phase-A-Experiment (2026-04-22 bis 2026-05-08)

`paper1_phaseA_v1` : 10 Systeme × 6 Varianten × 5 Seeds = **300 Läufe**, Laufzeit **4,5 Tage**, Ergebnis 300/300 erfolgreich.

Nebenbefund vom 2026-04-27, der später noch wichtig wird: `profile_init.jl` hing **über 48 Stunden** in einem einzelnen BFGS-Aufruf (Lorenz 3D). Reaktion war die Einführung von `time_limit_s = 300.0` als Wall-Clock-Notbremse im `BFGSoptimizer` — begründet mit „~100× der medianen Aufrufzeit, greift bei normalen Läufen nie“. Diese Annahme erwies sich knapp drei Monate später als falsch (siehe 3.8).

3.3 Auswertung und Freeze (2026-05-08 bis 2026-05-11)

Hypothesenverdikte:

Hypothese	Verdikt	Belegt auf
H1 Stage-Overshoot-Reduktion	PARTIAL	nur System 54 (1 von 6)
H2 kompetitive Rückgewinnungsqualität	SUPPORTED	7 von 8 Systemen
H3 Reduktion verschwendeter Level	PARTIAL	Systeme 11 und 54 (2 von 6)
H4 Ordnung der Usage-Policies	vakuös	alle <code>exact_match = 0</code> , Ordnung nicht unterscheidbar

Das stärkste Ergebnis war ein anderes — EvoGrow gegen die GP-Baseline auf gekoppelten Systemen:

System	Beste EvoGrow	GP	Faktor
31 SIR	7,0e-05	0,314	~4.500×
54 Lorenz	7,4e-04	0,921	~1.200×
26 Lotka-Volterra	2,5e-04	2,98e-03	~12×

Gleichzeitig eine bekannte Schwäche: System 11 ($du = -u^*$) erreicht Loss ~4,4e-15, aber `exact_match = 0` , weil Wachstum ohne Pruning Nullterme akkumuliert. Als echter algorithmischer Mangel dokumentiert, nicht als Metrikfehler.

Die Phase-A-Ergebnisse sind **eingefroren** und werden für die finalen Paper-Aussagen nicht mehr verwendet.

3.4 Strategischer Pivot (2026-05-17)

Paper 1 wird **kein** Pretuning-Vergleichspaper. Pretuning wird für Paper 1 vollständig deaktiviert (`use_pretuning = false`) und als mögliches Folgepaper zurückgestellt.

Neue Kernfrage: *Ist inkrementelles, gestuftes Wachstum ein effektiver Suchmechanismus für interpretierbare ODE-Entdeckung — und wo hilft es, wo versagt es?*

H1/H3 „PARTIAL“ wird umgedeutet: nicht als schwaches Ergebnis, sondern als **diagnostisches Signal**, dass systemweites Staging für mehrdimensionale Systeme zu grob sein könnte. Es wird eine Gate-Struktur eingeführt (Gate 1: Ist v2.2 paper-ready? Gate 2: Ist v3 paper-ready?).

3.5 Gate 1 — v2.2 scheitert (2026-05-30)

15 Diagnostikläufe, v2.2 `stage_local`, `use_pretuning = false` , `n_levels = 30` , 3 Seeds je System.

System	Ergebnis	Loss	Stage	Laufzeit
3 Logistic	<code>pruned_match = true</code> , alle Seeds	$\sim 7e-10$	Overshoot 0–3	—
11 Cubic	<code>pruned_match = true</code> , alle Seeds	$\sim 4e-15$	4/4	$\sim 1,3$ s
26 Lotka-Volterra	false , alle Seeds	$5e-4 - 1,4e-3$	5/3	2.800 – 9.400 s
31 SIR	false , alle Seeds	$7e-11 - 1e-4$	5/3	—
63 SEIR	false , alle Seeds	$9e-4 - 1,8e-3$	5/3	11.000 – 31.000 s

Auf System 26 lautet die gefundene Struktur $du_1 = 5,05 \cdot u_1 - 3,87 \cdot u_1 \cdot u_2$, $du_2 = 1,13 \cdot u_1 - 1,80 \cdot u_1^2$ — die Terme sind komplett falsch. Der Loss reißt in Stage 3 auf $\sim 1e-3$ auf, und danach verbessert die Eskalation bis Stage 5 nichts mehr; trigonometrische Terme sind hier nutzlos.

Diagnose: Der systemweite Staging-Mechanismus zwingt alle Gleichungen gemeinsam in höhere Stages, sobald der globale Fortschritt stagniert. Das ist kein Metrikfehler, sondern eine echte algorithmische Schwäche.

Entscheidung: v2.2 ist nicht paper-ready. Phase 2 (EvoGrow v3, gleichungsweises Staging) wird ausgelöst.

Nebenbeobachtung, noch offen: Auf System 31 Seed 42 wird bei Loss $\sim 7e-11$ ein Spurious-Term $0,0022 \cdot u_1$ nicht weggeprunt, weil die Schwelle $1e-3 \times \text{max_coeff} = 4e-4$ darunter liegt.

3.6 v3-Entwurf und Lockstep-Brücke (2026-07-20)

WP-v3.1 liefert `docs/evogrow_v3_design.md`: gleichungsweise Stage-Zustände (`eq_stages`, `eq_levels_in_stage`, `eq_plateau_histories`), ein Fortschrittssignal auf Basis des Ableitungsresiduums pro Gleichung, eine gleichungsweise Promotionsregel, gleichungsweise Kindergenerierung und neue Metriken.

WP-v3.2 implementiert `EvoGrowV3` bewusst als **Lockstep-Brücke**: Die Variante führt bereits gleichungsweise Zustände, promoviert aber alle Gleichungen gemeinsam und ist dadurch verhaltensgleich zu v2.2. Verifiziert als bit-identisch auf System 3 und 26. Das ist der Regressionsanker, gegen den alle späteren v3-Schritte geprüft werden.

3.7 Regressionshistorie (2026-07-20)

Auf Wunsch entstand eine **longitudinale Historie**, um zu verfolgen, ob Ergebnisse besser oder schlechter werden:

- `studies/regression/history.jsonl` — append-only, versioniert, ein Record je Zelle
- Schlüssel: (`git_hash`, `variant`, `system`, `seed`) plus `config_fingerprint` (SHA256 über die metrikrelevante Konfiguration, erste 16 Hex-Stellen)
- Suite: 5 Systeme (3, 11, 26, 31, 63) $\times$ 3 Seeds (42, 123, 7) $\times$ 2 Varianten = 30 Zellen

Dazu kam Fortschrittslogging: gestapelte Fortschrittsbalken (`ProgressMeter`), äußerer Balken über alle Zellen, innerer Balken über die Level eines Laufs, Detailausgabe nur in `run.log`. Leitlinie war ausdrücklich „so wenig wie möglich, aber so viel wie nötig“.

Ein Rechnerneustart beendete den ersten Volllauf nach 7 von 30 Records. Weil die Historie append-only ist, überlebten die 7. Daraufhin **WP-H1d (Resume)**: Der Runner überspringt beim Neustart alle erfolgreichen Zellen desselben `config_fingerprint`. Bewusst **nicht** auf `git_hash` geschlüsselt, damit ältere Records bei unveränderter Konfiguration weiterhin erkannt werden.

3.8 Der Kostenzusammenbruch (2026-07-22)

Der zweite Volllauf wurde nach **23 von 30 Zellen und 40,5 Compute-Stunden abgebrochen** — die Laufzeit war nicht mehr vertretbar.

Befund 1 — der Anker hält, mit einer Ausnahme. v2.2 und v3.2 waren in 7 von 8 überlappenden Zellen bit-identisch. Die Abweichung (System 26 Seed 123: 1,3916e-3 gegen 1,3713e-3) war **kein v3-Bug**, sondern ein Reproduzierbarkeitsleck: `BFGSOptimizer` wurde ohne `time_limit_s` konstruiert, der Default von 300 s Wall-Clock wurde an Optim.jl durchgereicht. Damit hing die Zahl der Optimierer-Iterationen von der Maschinenlast ab. Dieselbe Zelle brauchte 13.352 s gegen 20.158 s.

Befund 2 — das Kostenprofil. Rekonstruiert aus `run.log` über alle 23 Zellen:

Stage	Level	Zeit	Anteil	s/Level
1	124	0,7 h	1,9 %	21
2	129	6,1 h	16,3 %	170
3	94	11,6 h	30,9 %	443
4	55	7,4 h	19,7 %	482
5	48	11,7 h	31,3 %	878

Ein Stage-5-Level kostet das **42-fache** eines Stage-1-Levels. Pro Zelle oberhalb der erwarteten Stage aufsummiert:

24,9 von 40,5 Stunden — 62 % der gesamten Rechenzeit — wurden in Komplexität investiert, die die Systeme nie gebraucht haben.

Auf gekoppelten Systemen liegt der Anteil bei 38–80 %. Extremfall System 63 Seed 123: 8,4 h Laufzeit, davon 6,7 h verschwendet; ein einzelnes Level (16, Stage 4) kostete 3,4 Stunden.

Konsequenz: Laufzeitproblem und Gate-1-Failure-Mode sind **dasselbe Problem**. Die Suche läuft über die nötige Stage hinaus, und jede zusätzliche Stage ist überproportional teurer, weil kubische und trigonometrische Terme steife und divergierende Kandidaten-ODEs erzeugen.

Blocker für Phase B: 63 Systeme × 2 Bedingungen × 3 Seeds = 378 Läufe, davon 240 gekoppelt. Bei ~3,5 h je gekoppelter Zelle sind das **über 800 Stunden ≈ 5 Wochen durchgehend**. Phase B war mit diesem Kostenprofil nicht durchführbar.

3.9 WP-P1 — Determinismus, Budgets, Instrumentierung

Drei Runden (P1, P1b, P1c), jeweils mit Review:

- **Determinismus:** Wall-Clock aus dem Ergebnispfad entfernt (`time_limit_s = 86.400 s` als reine Notbremse, deren Auslösen gezählt wird).
- **Getrennte Budgets** für Screening während der Suche und finale Validierung, mit den heutigen Werten als Default, damit ohne Aktivierung kein Verhalten sich ändert.
- **Instrumentierung** pro Level: Fits, Integrationen, verworfene Integrationen, Zeitanteil Optimierung gegen Simulation, Retcode-Kategorien.

Zwei Reviewbefunde waren substanziell:

1. **isoutofdomain war das falsche Primitiv.** Belegt in den DiffEq-Quellen (`OrdinaryDiffEqCore/.../integrator_utils.jl:268–286`): Es verwirft den Schritt und verkleinert `dt`, statt abzubrechen — für divergierende Kandidaten also **teurer**. Das Abbruch-Primitiv ist `unstable_check`, dessen Default bereits `any(!isfinite, u)` prüft. Umgestellt.
2. **Der erste Benchmark-Zuschnitt hätte am Problem vorbeigemessen.** Mit 12 Leveln kostet System 26 nur 1,6 min; der Kostenausbruch beginnt bei Level 13 (147 s), 14 (878 s), 16 (611 s), 19 (1.484 s). Budget auf 18 Level korrigiert.

Ergebnis des Mikro-Benchmarks (System 26, Seed 42, 18 Level, je 370 Parameter-Fits):

	A Referenz	B Screening-Budgets	Faktor
Laufzeit	3.222,6 s	1.189,8 s	2,71×
Integrationen	1.741.484	2.488.973	0,70×
Kosten pro Integration	1,763 ms	0,409 ms	4,31×
Solve-Anteil an Laufzeit	95 %	86 %	
Overhead ohne Solve	153 s	166 s	

Sauberster Einzelvergleich: Stage 2, in beiden Fällen exakt 8 Level — A 2.937,9 s, B 297,1 s, **Faktor 9,9**.

Die Zahl, die alles Weitere bestimmt hat: 95 % der Laufzeit stecken in der ODE-Integration, der Overhead außerhalb beträgt 153 s. Ein Screening ohne Integration hätte damit eine Obergrenze von **~21×** (3.222,6 / 153).

Zwei Nebenfunde mit eigener Bedeutung:

- A ist bit-identisch zu Baseline v0 (`0.001391623174905009`) — aber v0 brauchte dafür 30 Level und 3,0 h und lief bis Stage 5, A erreicht denselben Loss in 18 Leveln und 53,7 min bei Stage 3. **Die Level 19–30 kosteten rund 2,1 h und verbesserten nichts.**
- Über alle 23 v0-Zellen: 13 liefen über Level 18 hinaus, und in **allen 13** war der Loss bei Level 18 bereits identisch zum Endergebnis. Das entspricht **15,8 von 40,5 h = 39 %** der Rechenzeit ohne jede Verbesserung.

Eine Kürzung des Level-Budgets wurde trotzdem abgelehnt. Der Loss bliebe identisch, `final_stage`, `stage_overshoot` und `wasted_levels` aber nicht: System 26 Seed 42 zeigt bei 30 Leveln Stage 5 / Overshoot 2, bei 18 Leveln Stage 3 / Overshoot 0. Ein Level-Cap würde das Overshoot-Phänomen **wegschneiden statt messen** — also genau das, was v3 beheben soll.

3.10 Die Screening-Spur (2026-07-22 bis 2026-07-23)

Idee: Die 95 % Solve-Anteil lassen sich nur angreifen, wenn die *Anzahl* der Integrationen sinkt. Kandidaten werden deshalb über ein **Ableitungsresiduum in geschlossener Form** bewertet (die Maschinerie liegt bereits in `pretune.jl`), und nur eine kleine Auswahl wird simuliert.

WP-P2.1 (Designnotiz) legte fest, dass Plateau-Erkennung, Stoplogik und Stage-Promotion weiterhin ausschließlich auf dem **simulierten** Loss arbeiten und der berichtete `loss` ein voll simulierter Loss bleibt. Das Review ergänzte das fehlende Kostenmodell:

Pro Level (20,6 Kandidaten)	Kosten	Faktor
heute: alle per BFGS mit Simulation	170,9 s	1,0×
Top 10 per BFGS	83,0 s	2,1×
Top 10, je 10 Polish-Iterationen	4,2 s	41×
Top 10, je 20 Polish-Iterationen	8,3 s	21×

Daraus die Architektur: **begrenztes Nachpolieren** statt roher LS-Parameter, weil `plateau_tol = 1e-4` und `loss_tol = 1e-8` auf BFGS-optimierte Losses kalibriert sind.

Der Verlauf war holprig und lehrreich:

1. **WP-P2.2/2b:** Drei Blocker im Review — eine Feldkollision, die Records unbrauchbar gemacht hätte; ein falscher Zähler für die Budget-Erschöpfung; und eine Rangübereinstimmung, die ihre eigene Frage nicht beantworten konnte, weil sie nur über die *ausgewählten* Kandidaten rechnete.
2. **WP-P2.2c (erste echte Zahlen, Toleranz 1e-6):** System 11 funktioniert (rho **+1,0**, 1,53× schneller, korrekte Struktur). System 3 versagt: rho **-0,78**, Loss 3,24e-8 statt 2,66e-10, Stage 5 statt 3, und mit 0,89× sogar **langsamer** als der Referenzpfad.
3. **WP-P2.3 (AIC):** Ergebnis bit-identisch zur Vorrunde, rho unverändert -7/9. Grund, im Nachhinein rechnerisch belegt: Bei $n = 200$ beträgt die AIC-Strafe über `p = 1..6` höchstens **10 Einheiten**, während der Fit-Term sich schon bei 10 % Residuenunterschied um **19** und bei Faktor 2 um **139** ändert. AIC war hier faktisch eine monotone Transformation des Residuums — und Spearman ist gegen monotone Transformationen invariant. BIC wäre mit $\leq 26,5$ ebenfalls zu schwach gewesen.
4. **Abbruch** per vorab vereinbarter Regel — und wenig später **Wiederaufnahme**, weil die Falsifikation weich war: Die Intervention hatte die Rangfolge gar nicht bewegt, die Hypothese war also ungeprüft, nicht widerlegt.

3.11 WP-T1 — die Numerik-Diagnose (2026-07-22)

Bevor der zweite Screening-Versuch lief, wurde ein Confounder geprüft: Der finale Refit kehrte nach **5 Loss-Auswertungen** mit `retcode = Success` zurück, ohne etwas zu verbessern. Vermutung: Der Bewertungspfad simuliert mit `abstol = reltol = 1e-6` — Gradienten aus finiten Differenzen einer Größe, die nur auf 1e-6 genau berechnet wird, wären in diesem Bereich Rauschen.

Teil A — bestmöglich erreichbarer Loss mit den wahren Parametern:

Toleranz	System 3	System 11
1e-5	2,06e-10	4,79e-13
1e-6	4,40e-12	1,86e-14
1e-8	6,38e-16	1,36e-17
1e-10	4,58e-17	3,84e-19
1e-12	4,34e-17	3,86e-19

Baseline v0 meldet für System 11 einen Loss von $4,402e-15$ — besser als das, was die wahren Parameter bei dieser Toleranz erreichen können ($1,86e-14$). Diese Zahl ist numerisches Rauschen.

Sie steht in Baseline v0, in der Phase-A-Auswertung und in allen bisherigen Regressionsprüfungen.

Teil B — Rauschgrenze des Optimierers. Erreichter Loss aus dem LS-Warmstart:

Toleranz	System 3	System 11
1e-6	3,236e-08	4,606e-15
1e-8	3,236e-08	1,669e-17
1e-10	3,236e-08	4,860e-18
1e-12	3,236e-08	4,856e-18

System 11 bestätigt die Vermutung (der Wert skaliert direkt mit der Toleranz). **System 3 widerlegt sie** — dort ist der Wert über sechs Größenordnungen völlig flach, obwohl der numerische Boden bei $4,40e-12$ liegt, also 7.000-fach darunter.

Und damit war die eigentliche Ursache gefunden: $3,236e-08$ ist exakt der Wert, bei dem die Screening-Variante hängenblieb. Der Referenzpfad erreicht $2,66e-10$ und benutzt **keinen** Warmstart (`use_pretuning = false`). Der ableitungsbasierte Warmstart führt auf System 3 in ein Becken, aus dem BFGS nicht herausfindet — die Screening-Variante hatte also genau das wieder eingeführt, was die Konfiguration bewusst abgeschaltet hatte.

Zwei weitere Befunde nebenbei:

- **Pathologische Line-Search:** Einzelne Fits verbrauchen bei **zwei Parametern** bis zu **39.933 Loss-Auswertungen** (jede eine vollständige Integration) mit Retcode `Failure`, Laufzeiten 3,2 / 10,4 / 23,6 s — bei `maxiters = 200`, also rund 200 Auswertungen pro Iteration. Das erklärt die 7.281 Integrationen pro Fit aus der Regressionsmessung.
- **Sentinel-Loss:** Ein vollständig gescheiterter Fit meldet `final_loss = 1.000e+06` (der Initialwert `l_best`) mit Retcode `Success` und ist von einem echten schlechten Fit nicht unterscheidbar.

3.12 WP-P2.4 — der Durchbruch (2026-07-23)

Zwei getrennte Interventionen, jede einzeln messbar:

1. **Geschachtelter Modellvergleich als Gate** statt additivem Strafterm. Begründung: Kinder sind geschachtelte Obermengen ihrer Eltern, und für geschachtelte Least-Squares-Probleme ist das Residuum monoton nicht-steigend in der Termzahl — ein größeres Modell kann nie schlechter abschneiden. Ein Kind darf seinen Elternteil nur überholen, wenn die Residuenverbesserung größer ist, als ein zusätzlicher Parameter zufällig liefern würde.
2. **Polish-Start vom Screening entkoppelt:** Der LS-Fit dient nur noch der Bewertung, der Startpunkt fürs Nachpolieren kommt vom Referenzpfad.

System 3, Bewertungstoleranz 1e-8:

Bedingung	Zeit	Loss	Stage	Fits	Integrationen
A Referenz	78,6 s	6,25e-9	2	110	275.098
B Residuum + LS-Start (<i>vorher</i>)	353,2 s	3,236e-8	5	241	761.581
C Nested-Gate + LS-Start	49,2 s	3,236e-8	5	241	232.224
D Nested-Gate + entkoppelter Start	12,6 s	2,558e-9	2	61	73.680

D schlägt den Referenzpfad um Faktor 6,2 bei besserem Loss und korrekter Stage.

System 11: alle Bedingungen qualitativ gleichwertig (Loss $\sim 2,0e-16$, Stage 4, `pruned_match` true), D mit 0,6 s gegen 1,1 s am schnellsten.

Die Arbeitsteilung ist sauber getrennt:

- Das **Gate** senkt Kosten und repariert das Ranking: B $\rightarrow$ C bringt 7,2 $\times$ bei identischem Ergebnis, 147 von 400 Kindern scheitern am Gate, die Auswahl weicht in 126 Fällen auf 7 von 20 Leveln ab, und die Rangübereinstimmung kippt von **-0,78 (Median -1,0)** auf **+0,26 (Median +0,48)**.
- Der **entkoppelte Startpunkt** repariert Qualität und Eskalation: C $\rightarrow$ D.

Keine der beiden Interventionen allein hätte gereicht.

Der Nebenfund mit möglicherweise größerer Tragweite: Der **Referenzpfad allein** profitiert schon von der engeren Toleranz.

System 3, Referenzpfad	Zeit	Loss	Stage	Overshoot
bei Toleranz 1e-6	279,3 s	2,66e-10	3	1
bei Toleranz 1e-8	78,6 s	6,25e-9	2	0

Erklärung: Bei 1e-6 erreicht der Optimierer auf Stage 2 die Schwelle `loss_tol = 1e-8` nicht zuverlässig, die Abbruchbedingung feuert nicht, und die Suche eskaliert.

Ein Teil des beobachteten Stage-Overshoots wäre damit ein numerisches Artefakt und keine Eigenschaft der Promotionsregel. Das berührt unmittelbar die 62-Prozent-Rechnung und die Begründung für v3. Bislang: eine Zelle, ein Seed, ein 1D-System — eine Hypothese, kein Befund.

4. Was implementiert ist und funktioniert

Komponente	Status
<code>discover()</code> Kernpipeline	stabil seit 2026-04-20
EvoGrow v1, v2.1, v2.2	stabil
<code>EvoGrowV3</code> (Lockstep-Brücke)	implementiert, bit-identisch zu v2.2 verifiziert
<code>EvoGrowScreening</code>	implementiert, Nested-Gate und entkoppelter Start
<code>GPStructureSearch</code>	Vergleichsbaseline
<code>StagedPolynomialBasis</code> , <code>PolynomialBasis</code>	stabil
<code>BFGSOptimizer</code>	+ deterministische Budgets, frühe Verwerfung, Retcode-Kategorien
<code>pretune.jl</code>	+ <code>derivative_screening_diagnostics</code> mit Gültigkeitsflag
Experimentinfrastruktur	Manifest, Runner, Aggregator, atomare Writes
Regressionshistorie	append-only, Fingerprint-basiert, resume-fähig
Kosteninstrumentierung	pro Level und pro Lauf, bis in die Records
Mikro-Benchmark	<code>studies/profiling/profile_eval_cost.jl</code>
Vergleichsskript Screening	<code>studies/debug/compare_screening_variant.jl</code> , vier Bedingungen
Numerik-Diagnose	<code>studies/numerics/solver_tolerance_noise_floor.jl</code>

5. Was verworfen wurde — und warum

Verworfen	Grund	Beleg
Pretuning als Paper-1-Thema	Strategischer Fokuswechsel auf gestuftes Wachstum	2026-05-17
v2.2 als finale Paper-Variante	Systemweites Staging versagt auf gekoppelten Systemen	Gate 1, 3 von 5 Systemen <code>pruned_match = false</code>
<code>isoutofdomain</code> für frühe Verwerfung	Verwirft den Schritt und verkleinert <code>dt</code> , bricht nicht ab — teurer statt billiger	DiffEq-Quellen, <code>integrator_utils.jl:268–286</code>
Level-Budget-Kürzung auf 18	Loss bliebe identisch, aber Overshoot-Metriken würden verfälscht	Stage 5/Overshoot 2 bei 30 gegen Stage 3/Overshoot 0 bei 18 Leveln
AIC als Komplexitätsstrafe	Bei $n = 200$ vom Fit-Term dominiert; Strafe ≤ 10 gegen Fit-Differenz 19–139	WP-P2.3, Ergebnisse bit-identisch
BIC als Alternative	Ebenfalls zu schwach ($p \cdot \log n \leq 26,5$)	Rechnerisch, nicht getestet
LS-Warmstart als Polish-Startpunkt	Führt auf System 3 in ein Becken bei $3,236e-08$	WP-T1, flach über sechs Toleranz-Größenordnungen
Rangübereinstimmung nur über ausgewählte Kandidaten	Kann die eigene Frage nicht beantworten	WP-P2.2b-Review
Screening-Spur (zwischenzeitlich)	Abbruchregel ausgelöst — später revidiert, weil die Falsifikation weich war	2026-07-22, wieder aufgenommen 2026-07-23

6. Die wichtigsten Zahlen an einem Ort

Kostenprofil (23 Zellen, 40,5 h): Stage 1 → 21 s/Level, Stage 5 → 878 s/Level (Faktor 42). 62 % der Rechenzeit oberhalb der nötigen Stage. 39 % nach dem Punkt, an dem der Loss bereits final war.

Bewertungspfad (System 26): 95 % Solve-Anteil, 4.707 Integrationen pro Parameter-Fit, 1,763 ms pro Integration, 153 s Overhead außerhalb der Integration.

Erreichte Beschleunigungen:

Maßnahme	Faktor	Gemessen auf
Solver-Budgets (WP-P1b)	2,71×	System 26, 370 Fits je Bedingung
Solver-Budgets, nur Stage 2	9,9×	System 26, je 8 Level
Nested-Gate (B → C)	7,2×	System 3
Bedingung D gegen Referenz	6,2×	System 3
Toleranz 1e-6 → 1e-8, Referenzpfad allein	3,6×	System 3

Numerische Grenzen (bestmöglicher Loss mit wahren Parametern): bei 1e-6 System 3 → 4,40e-12, System 11 → 1,86e-14. Der in Baseline v0 berichtete System-11-Loss von 4,402e-15 liegt darunter.

7. Offene Punkte, nach Dringlichkeit

1. **Ist der Stage-Overshoot teilweise numerisch?** Die stärkste offene Frage. Auf System 3 verschwindet der Overshoot allein durch die engere Toleranz. Zu prüfen auf **System 26**, wo Gate 1 gescheitert ist. Berührt direkt die Begründung für v3.
2. **Bewertungstoleranz dauerhaft auf 1e-8?** Kosten ~1,3× je Fit, aber der Referenzpfad wurde in der Messung netto 3,6× schneller, weil weniger Level nötig waren. Betrifft `config_fingerprint` und erfordert eine neue Baseline.
3. **Belastbarkeit bisheriger Losses.** Der System-11-Wert ist Rauschen. Zu klären, welche weiteren berichteten Zahlen betroffen sind — auch in Phase A.
4. **Neue Regressionsbaseline** unter der aktuellen Konfiguration. Baseline v0 (`config_fingerprint 0c739d4e36ee6498`) bleibt als historischer Datensatz gültig, taugt aber nicht mehr als Vergleich.
5. **WP-v3.3** (gleichungsweise Kindergenerierung) ist spezifiziert und wartet. Verhaltensneutraler Umbau — muss bit-identisch bleiben, solange alle Gleichungen im Lockstep promovieren.
6. **Pathologische Line-Search** (bis zu 39.933 Auswertungen bei zwei Parametern) — unangetastet, aber ein erheblicher Kostenhebel.
7. **Sentinel-Loss 1e6 mit Retcode Success** — gescheiterte Fits sind nicht als solche erkennbar.
8. **Pruning-Schwelle** $1e-3 \times \max_coeff$ möglicherweise zu streng (System 31 Seed 42, Spurious-Term `0,0022 · u1`).

8. Der rote Faden

Das Projekt hat drei Ebenen, die sich in den letzten Tagen als verschränkt erwiesen haben:

Ebene 1 — die Forschungsfrage. Ist gestuftes, inkrementelles Wachstum ein guter Suchmechanismus? Gate 1 hat gezeigt: für gekoppelte Systeme in der v2.2-Form nicht. Daraus entstand v3 (gleichungsweises Staging). Diese Linie ist unverändert gültig und wartet bei WP-v3.3.

Ebene 2 — die Kosten. Die Suche war nicht nur langsam, sondern *falsch* langsam: 62 % der Rechenzeit gingen in Komplexität, die kein System brauchte. Overshoot und Kosten sind dasselbe Problem, weil jede zusätzliche Stage überproportional teurer ist. Das macht v3 zugleich zum Performance-Fix.

Ebene 3 — die Numerik. Erst zuletzt wurde sichtbar, dass ein Teil der beobachteten Effekte möglicherweise gar nicht algorithmisch ist. Ein berichteter Loss lag unterhalb der Rechengenauigkeit. Ein Optimierer meldete Konvergenz, wo er nur blind war. Und die engere Toleranz allein hat auf einem System den Overshoot beseitigt.

Was das für die nächsten Schritte heißt: Ebene 3 muss zuerst geklärt werden, weil sie die Messgrundlage der beiden anderen betrifft. Die entscheidende Einzelmessung ist der Toleranzvergleich auf System 26 — dem System, an dem Gate 1 gescheitert ist. Fällt der Overshoot dort ebenfalls, ist ein Teil der v3-Begründung neu zu bewerten. Bleibt er, ist v3 bestätigt und wir haben die Numerik nebenbei repariert.

In beiden Fällen ist das Ergebnis verwertbar. Das ist die angenehme Eigenschaft dieser Messung.